

FUN22 unwinnability — specification

ashlar-chess · governing document for the
 io.github.dlbbld.ashlarchess.unwinnability engine

This document is the **governing specification** for ashlar’s unwinnability engine (the `io.github.dlbbld.ashlar` package internals). It is derived **only** from Miguel Ambrona’s FUN 2022 paper, “*A Practical Algorithm for Chess Unwinnability*” — the **full version** (the one with complete proofs of Lemmas 10 and 11). Neither ashlar’s former `cha` port nor Ambrona’s D3-Chess/`cha/chasolver` source was consulted. Review the engine code against this document (and the paper), not against any other codebase.

Sections 1–5 specify the semi-static algorithm (Theorem 12) in full detail; Section 6 maps the remaining paper figures (the search side) to their implementing classes and records their footnote treatments; Section 7 records what ashlar deliberately adds on top of the paper. Section/figure/line references point at the full version unless noted.

1. Scope

The heart of the engine is **Theorem 12**:

$$\text{UnwinnableStatic}(pos, c, \text{Mobility}(pos)) = \text{true} \implies pos \text{ is unwinnable for } c,$$

i.e. the composition of the **mobility over-approximation** (§3.1, Fig. 6–7, Lemma 8, Corollary 9) with the **semi-static check** (§3.2, Fig. 8, Lemmas 10–11). The semi-static check is sound and deliberately incomplete: it may decline on truly-unwinnable positions, but a true must never be wrong.

Around it, the paper’s full algorithm (Fig. 9 = semi-static shortcut + Find-Helpmate search under iterative deepening) and quick algorithm (Fig. 10) are implemented per Section 6.

2. Board model and notation

Squares are integers 0..63 with $a1 = 0$, $b1 = 1$, ..., $h8 = 63$ (little-endian rank-file; the ashlar/Stockfish layout — $\text{file}(s) = s \& 7$, $\text{rank}(s) = s \gg 3$, coinciding with `Square.ordinal()`).

A **piece** P is $(type, color, sq)$ with $type \in \{K, Q, R, B, N, P\}$, $color \in \{w, b\}$, $sq \in 0..63$. A **position** pos is a set of pieces on distinct squares (`SemiStaticPosition/SemiStaticPiece`). $\text{color}(s)$ is the light/dark colour of square s .

Per-square geometric sets (all “over an empty board”; implemented in `SquareGeometry`):

set	meaning
$\alpha(s)$	orthogonally adjacent squares (share a border; opposite square colour)
$\beta(s)$	diagonally adjacent squares (same square colour)
$\delta(s) = \alpha(s) \cup \beta(s)$	all ≤ 8 king-neighbours — the king’s escape squares
$\nu(s)$	squares at knight-distance 1
$\omega_w(s) / \omega_b(s)$	the ≤ 1 square from which a white / black pawn reaches s by one non-capturing push
$\pi_w(s) / \pi_b(s)$	the squares from which a white / black pawn attacks s

Double pawn pushes need no special case: reachability of a rank-4 square is already implied stepwise through the rank-3 square, and admissibility (§3) makes this over-approximation sound.

2.1 Predecessors (Fig. 6 preamble, full version 1.523–555)

Implemented in **Predecessors**. $\text{pred}_P(s)$ — squares from which $P.type$ reaches s in one non-capture move:

$$\text{pred}_P(s) = \begin{cases} \nu(s) & \text{if } P.type = N \\ \beta(s) & \text{if } P.type = B \\ \alpha(s) & \text{if } P.type = R \\ \delta(s) & \text{if } P.type \in \{Q, K\} \\ \omega_{P.color}(s) & \text{if } P.type = P \end{cases}$$

Note sliders use only the **adjacent** squares ($\beta/\alpha/\delta$). Long slides emerge stepwise from the Fig. 7 fixpoint; enemy blockers are (soundly) ignored.

$\text{pred-capt}_P(s)$ — squares from which P captures onto s :

$$\text{pred-capt}_P(s) = \begin{cases} \pi_{P.color}(s) & \text{if } P.type = P \\ \text{pred}_P(s) & \text{otherwise} \end{cases}$$

$\text{prom}(P)$ — promotion squares, defined only for pawns:

$$\text{prom}(P) = \begin{cases} \{a8..h8\} & \text{if } P.color = w \\ \{a1..h1\} & \text{if } P.color = b \\ \emptyset & \text{otherwise} \end{cases}$$

$\text{attackers}(s) = \{P \in pos : P.sq \in \text{pred-capt}_P(s)\}$ — pieces currently attacking s .

3. Mobility (Fig. 6 rules + Fig. 7 fixpoint)

Implemented in **Mobility/MobilitySolution**. Boolean variables over the current position:

- $M[P][s]$ — piece P can *eventually* move to square s .
- $R[s][c]$ — square s can eventually be reached (or is currently occupied) by a **non-king** piece of colour c .
- $C[P]$ — piece P can be cleared from $P.sq$ (by moving away or being captured).

3.1 Figure 6 — the implications (heads may repeat)

For a variable V , V becomes **1** iff the body of *every* rule with head V is **true** (conjunction across rules; each body may itself be a disjunction/conjunction). All literals are positive $\Rightarrow$ the system is monotone.

Move (non-pawn P , $s \neq P.sq$):

$$M[P][s] \Leftarrow \bigvee_{u \in \text{pred}_P(s)} M[P][u]$$

Pawn move ($P.type = P, s \neq P.sq$). Let $F_s^c := C[P']$ if some $P' \in pos$ has $P'.color = c$ and $P'.sq = s$, else $F_s^c := \text{true}$:

$$\begin{aligned}
M[P][s] \Leftarrow & \left(\bigvee_{u \in \text{pred}_P(s)} M[P][u] \right) \wedge F_s^{\neg P.color} && \text{push (enemy on } s \text{ clearable)} \\
\vee & \left(\bigvee_{u \in \text{pred-capt}_P(s)} M[P][u] \right) \wedge R[s][\neg P.color] && \text{capture (enemy reaches } s) \\
\vee & \left(\bigvee_{u \in \text{prom}(P)} M[P][u] \right) && \text{reached a promo sq} \Rightarrow \text{go anywhere}
\end{aligned}$$

(The push's *own-colour* blocker on s is handled by the separate **not self-capture** rule below, so F here guards the *enemy* on the push target; see §5 note P1.)

Clearance ($P \in pos$):

$$C[P] \Leftarrow \left(\bigvee_{s \neq P.sq} M[P][s] \right) \vee \left(\bigvee_{\substack{P' \in pos \\ P'.color \neq P.color}} M[P'][P.sq] \right)$$

Reachability ($s \in S, c \in \{w, b\}$):

$$R[s][c] \Leftarrow \bigvee_{\substack{P \in pos, P.color=c \\ P.type \neq K}} M[P][s]$$

King attackers ($P.type = K, s \neq P.sq$):

$$M[P][s] \Leftarrow \bigwedge_{\substack{P' \in \text{attackers}(s) \\ P'.color \neq P.color}} C[P']$$

Not self-capture ($P, P' \in pos, P \neq P', P.color = P'.color$):

$$M[P][P'.sq] \Leftarrow C[P']$$

A king thus needs **move** $\wedge$ **king-attackers** $\wedge$ (**not-self-capture if ally on** s) all true; a pawn needs **pawn-move** $\wedge$ (**not-self-capture if ally on** s).

3.2 Figure 7 — the fixpoint

1. $M[P][s] = C[P] = R[s][c] = 0$ for all P, s, c .
2. $M[P][P.sq] = 1$ for all $P \in pos$ (a piece can “move” to its own square).
3. repeat: for every variable V still 0, if every Fig-6 rule with head V has a true body on the current state, set $V = 1$;
4. until no variable changes;
5. return $\{ M[P][s] \}$.

Monotone, so it converges; the least such fixpoint is **admissible**: $M[P][s] \geq M^*[P][s]$ where M^* is the true mobility (Lemma 8 / Corollary 9). Over-approximation is safe because the semi-static check is monotone-decreasing in M (Lemma 10).

4. UnwinnableStatic (Fig. 8)

Implemented in `UnwinnableSemiStatic`. Input pos , intended winner c , mobility M . Output `bool`.

1. if en passant is possible OR any player has castling rights $\rightarrow$ return `false`.
2. $\text{region}(P) := \{s \mid M[P][s] = 1\}$ for each $P \in pos$.
3. $K_c := \text{winner's king}$; $K_{-c} := \text{loser's king}$.
4. $\text{intruders} := \{P \in pos \mid P.\text{color} = c \wedge \text{region}(P) \cap \text{region}(K_{-c}) \neq \emptyset\}$.
5. if $\exists P \in \text{intruders}$ with $P.\text{type} \neq B \rightarrow$ return `false`.
6. if $\exists P, P' \in \text{intruders}$ with $\text{color}(P.sq) \neq \text{color}(P'.sq) \rightarrow$ return `false`.
7. $\text{att-region}(P) := \{s \mid \text{pred-capt}_P(s) \cap \text{region}(P) \neq \emptyset\}$ for each $P \in pos$.
8. $\text{blockers}(s) := \{P \in pos \mid P.\text{color} \neq c \wedge P.\text{type} \neq K \wedge \text{region}(P) \cap \alpha(s) \neq \emptyset\}$
(loser's non-king pieces — occupy a bordering square).
9. $\text{assistants}(s) := \{P \in pos \mid P.\text{color} = c \wedge \text{att-region}(P) \cap \alpha(s) \neq \emptyset\}$
(winner's pieces — attack a bordering square).
10. if $\exists s \in \text{region}(K_{-c})$ such that

$$|\text{blockers}(s)| + |\text{assistants}(s)| \geq |\alpha(s)| \quad \text{AND} \quad \exists P \in pos \text{ with } s \in \text{att-region}(P) \wedge P.\text{color} = c$$
 $\rightarrow$ return `false`.
11. return `true` (position is unwinnable for c).

Reading (§3.2, Lemma 11): a position is unwinnable if for **every** square either (i) the loser's king can't reach it, or (ii) the winner can't attack it, or (iii) its **bordering squares** $\alpha(s)$ can't all be simultaneously blocked (by a loser non-king piece occupying — the king can't capture its own piece) or covered (by a winner piece attacking). Step 10 returns `false` as soon as one square fails all three — i.e. looks matable. Steps 5–6 enforce Lemma 11's precondition: the only pieces that may enter the loser-king region (hence deliver check) are same-coloured bishops (or none). In genuinely blocked positions the two kings' regions are disjoint, so the winner's king is *not* an intruder and step 5 does not fire.

Why $\alpha(s)$ and not all 8 neighbours $\delta(s)$ (verified on the 500-dpi render: steps 8–10 carry the paper's share-a-border glyph): the $\alpha(s)$ squares have the *opposite* square colour to s , so once steps 5–6 guarantee that only same-coloured bishops can enter the loser-king region, no bishop intruder can ever attack an $\alpha(s)$ square — the only possible assistants there are pawns or the winner's king, and each of those covers at most one $\alpha(s)$ square (Lemma 11 footnote). That is exactly what makes the step-10 *count* a sound proxy for simultaneous coverage. Figure 8's own footnote *a* confirms the restriction: “we could design a more complete check that looks at **all** neighbours of s , but the condition on step 10 would be significantly more involved (to ensure monotonicity).” A δ -based count is NOT covered by the paper's proof (a single piece can cover several δ squares at once, so the count would under-estimate coverage and could wrongly conclude unwinnability).

Soundness direction. Every approximation widens the “possibly matable” set: M over-approximates reach (step 5/10 fire more easily), and step 10 counts *pieces* vs. $\alpha(s)$ squares (each blocker/assistant covers ≤ 1 of them under the bishop precondition — Lemma 11 footnote — so the count over-estimates coverage). Over-estimating matability can only turn a true UNWINNABLE into `false` (UNKNOWN), never the reverse. Hence `true` is always correct = sound.

5. Theorem 12

Theorem 1 (Theorem 12). $\text{UnwinnableStatic}(pos, c, \text{Mobility}(pos)) = \text{true} \implies pos \text{ is unwinnable for } c.$

Proof. $M \leftarrow \text{Mobility}(pos)$ is admissible (Corollary 9); UnwinnableStatic is monotone decreasing in M (Lemma 10) and sound on the true mobility M^* (Lemma 11). Since $M \geq M^*$,

$\text{UnwinnableStatic}(pos, c, M) = \text{true} \implies \text{UnwinnableStatic}(pos, c, M^*) = \text{true} \implies pos \text{ unwinnable for } c.$ $\square$

Precondition (step 1): the paper proves Lemma 8 only for positions **without castling rights and without a possible en passant**; those cases return false (unknown) up front.

6. Resolved OCR ambiguities (audit trail)

The paper’s figures use $\neq \emptyset \cap \notin \geq$ glyphs that do not survive text extraction. Each was resolved from the full version’s inline prose / proofs, not guessed:

- **Step 5** $P.type \neq B$: full-version note — “the set of intruders [must] be empty or formed entirely by bishops.”
- **Step 6** $\text{color}(P.sq) \neq \text{color}(P'.sq)$: note — “all intruders ... be of the same square color.”
- **Step 8** $P.color \neq c$ (loser’s non-king pieces), **step 9** $P.color = c$ (winner’s pieces): the two differ in side. The rendered Figure 8 annotates step 8 as “the intended loser’s pieces that can potentially block an adjacent square to s ” — the loser king cannot capture its own piece, so only a loser-owned piece is a reliable escape blocker; the winner’s pieces instead attack escape squares (step 9). (An earlier draft wrongly set step 8 to $= c$ after a corrupted text extraction showed $= c$ on both lines; corrected from the rendered figure — a soundness fix, since counting winner pieces here would undercount loser-owned blockers and could return UNWINNABLE for a matable position.)
- **Steps 8–10 neighbour set** $= \alpha(s)$, **not** $\delta(s)$ (fixed 2026-07-03): at 220 dpi the share-a-border glyph was misread as the 8-neighbour glyph by two independent reviewers; the 500-dpi zoom is unambiguous, and both figure footnote *a* and the Lemma 11 footnote only make sense for α . An earlier draft (and implementation) used $\delta(s)$ with threshold $|\delta(s)|$ — empirically it produced zero contradictions over the corpus, but it is not covered by the paper’s soundness proof. Lesson: verify glyph-level figure details at high DPI.
- **Note P1 — pawn push F factor** (verified against the rendered figure): the paper defines F_s^c via the piece of colour $\neq c$ on s and uses $F_s^{P.color}$; this spec defines F_s^c via the piece of colour $= c$ and uses $F_s^{\neg P.color}$. Both denote exactly “the enemy piece on the pawn’s push target s can be cleared,” matching the prose “a possibly **enemy** piece on the target square can be cleared first,” and the own-colour blocker is already handled by **not self-capture**. Either sign is a *valid* (sound) constraint — it only removes genuinely impossible mobility, preserving admissibility $M \geq M^*$ — so this choice cannot break soundness; it is documented for fidelity.

7. The search side (Figures 5, 9, 10, 12, 13)

Implemented independently from the same paper, driven over ashlar’s mutable `HelpmateSearchBoard` hot path (twelve mutable bitboards, make/unmake, per-depth legal-move buffers, cached terminal flags — chess foundation, not unwinnability logic; lock-step-tested against the public Board):

- **Figure 5 Find-Helpmate** → `FindHelpmate`: depth- and node-bounded DFS for a checkmate *by the intended winner*, with a depth-aware transposition table and the Figure 12 Score budget adjustment. Terminal leaves: checkmate, stalemate, the winner’s bare king, and the Lemma 5/6 material shapes (`MaterialLemmas`). **Footnote b** (implemented; resolves deep helpmates like K+Q at small bounds): a Normal-scored move directly following a Reward-scored move is also rewarded. Footnote a (Stockfish move ordering) is deliberately not implemented — perf-only.
- **Figure 9 full routine** → `UnwinnableFullAnalyzer`: semi-static shortcut, then iterative deepening over Find-Helpmate; `WINNABLE` on an exhibited helpmate, `UNWINNABLE` on an un-interrupted exhaustion, else `UNDETERMINED` when the budget runs out. The paper leaves `bound(d)` as a parameter (its practical note, text footnote 13, fixes a small constant); ashlar uses one global 500 000-node budget across iterations with a depth ceiling of 100 (each iteration’s node bound is the remaining global budget). **The transposition table is per-iteration — a deliberate deviation from the paper’s prose, which says the table “can be shared between different calls”**. The soundness of Figure 9 step 5–6 (“search not interrupted ⇒ Unwinnable”) rests on the interrupted flag being monotone over every visit the table’s entries summarize: within one iteration, an entry written by a depth-cut visit coexists with the raised flag, so that iteration can no longer claim `UNWINNABLE`. A *stale* entry from an earlier, depth-cut iteration would prune a later iteration’s node *without re-raising its interrupt flag* — the later iteration could believe it exhausted the tree while cut lines hide behind the stale prune, a potential false `UNWINNABLE`. The absence-of-mate half of a stale entry is still a true statement; it is the exhaustion *witness* that does not survive sharing. The search-state key includes the footnote-b reward-chain flag: a visit with the boost pending explores a strictly stronger budget shape than one without, so the two states must not prune each other.
- **Figure 10 quick routine** → `UnwinnableQuickAnalyzer`: forced-move advance, bounded DFS with a *global* depth-9 interrupt, then the semi-static check gated to pawn/bishop/king material without semi-open files. **Footnote a** (implemented): the forced-move advance is loop-guarded (arbitrarily long single-move sequences exist; capping is verdict-preserving since a forced prefix is equivalence-preserving). **Footnote b** (implemented): the Lemma 5/6 material positions are DFS leaves — that is what makes not-interrupted ⇒ `UNWINNABLE` fire on imminently terminating positions.
- **Figure 12 Score** → `Score`: Normal/Reward/Punish move classification; pure efficiency heuristic.
- **Figure 13 Going-to-corner** → `GoingToCorner`: rewards slow-piece (K/N) moves approaching the mating corner; the depth extension that lets bounded search finish long mating plans.
- **Lemmas 5/6** → `MaterialLemmas`: winner-side insufficient-material shapes (strict: *exactly one* knight; bishops of *one* square colour), plus the Figure 12 material condition (same shapes without the pawn-freeness gate).

Text footnote 4 (transposition cut) and footnote 13 (constant practical bound) are honoured as above. Text footnote 12 notes that `cha` evolved beyond the paper with extra heuristics —

divergence between this engine and cha-lineage implementations on *completeness* (who proves more) is therefore expected; divergence on *soundness* is a bug.

8. Ashlar extensions on top of the paper

Deliberate, clearly-layered additions — none alters the paper algorithms' verdict logic:

- **Mate line** (`UnwinnabilityFullAnalysis.mateLine()`): the Figure 5 search records the exhibited helpmate line on the unwind — bookkeeping only. A `WINNABLE` verdict is always search-proven and carries its witnessing line (empty exactly for a zero-move helpmate, i.e. the submitted position is already the intended winner's checkmate).